

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
b = fun()
```

```
print(a)
print(b)
```

- A)Error
- B)[10]
- [10]
- C)[10]
- [10, 10]
- D)[10, 10]
- [10]

Q2. What happens here?

```
class MyError(Exception):
    pass
```

```
def test():
    raise MyError("Something went wrong")
```

```
try:
    test()
except MyError as e:
    print(type(e).__name__)
```

- A)TypeError
- B)Something went wrong
- C)MyError
- D)Exception

Q3. What will happen?

```
def outer():
    x = 10
```

```
    def inner():
        nonlocal y
```

y = 20

```
    inner()
```

```
outer()
```

- A)10
- B)NameError
- C)SyntaxError
- D)20

Q4. What will normally happen?

```
import math
import math
```

- A)SyntaxError
- B)ImportError
- C)The module is normally loaded and executed twice
- D)The module is normally loaded once and reused

Q5. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

- A)B
- B)A, and execution ends normally
- C)A, followed by propagation of the same exception
- D)No Output

Q6. What happens when this function is called?

```
def fun(a=10, b):
    return a + b
```

- A)NameError
- B)It executes correctly
- C)TypeError
- D)SyntaxError

Q7. What does the following function return?



```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)None
- B)Hello
- C)Error
- D)Hello None

Q8. Which object keeps track of modules that have already been imported during the current Python process?

- A)sys.modules
- B)sys.imports
- C)sys.path
- D)__module__

Q9. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

```
print(fun(-5))
```

- A)negative
cleanup
- B)100
cleanup
- C)cleanup
-5
- D)cleanup
100

Q10. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")
```

```
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

```
print(fun())
```

- A)A
- B
- D
- 10
- B)A
- B
- C
- D
- 10
- C)A
- B
- 10
- D
- D)A
- C
- D
- 20

Q11. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

```
print(fun())
```

- A)None
- B)20
- C)RuntimeError
- D)10

Q12. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)6
- B)6.0
- C)NameError
- D)36



Q13. What is the output?

```
try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

- A)B
- B)A C
- C)A
- D)B C

Q14. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

- A)10
- B)-10
- C)30
- D)20

Q15. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It prevents all imports
- B)It prevents syntax errors
- C)It identifies code intended to run when the file is executed directly
- D)It creates the main function

Q16. What will be the output?

```
x = 10
```

```
def fun():
    print(x)
    x = 20
```

```
fun()
```

- A)10
- B)UnboundLocalError
- C)NameError
- D)20

Q17. What happens when this code executes?

```
def fun():
    print(variable)
    variable = 50
```

```
variable = 10
```

```
fun()
```

- A)10
- B)NameError
- C)50
- D)UnboundLocalError

Q18. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
            numbers)
    )
)
```

```
print(result)
```

- A)[2, 4]
- B)[1, 3, 5]
- C)[2, 4, 6, 8, 10]
- D)[4, 8]

Q19. What happens here?

```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

- A)e
- B)Nothing
- C)Wrong Value
- D)ValueError

Q20. if test.py has print("A")

```
if __name__ == "__main__":
    print("B")
```

If test.py is executed directly, what will be printed?

- A)A
- B)B
- C)A followed by B



D)Nothing

Q21. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)InvalidAgeError

15

B)Exception

Age is too low

C)InvalidAgeError

Age is too low

D)Nothing, because custom exceptions cannot be caught

Q22. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

A)30

20

B)20

30

C)20

10

D)30

30

Q23. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

A)random cannot be imported

B)Python will always reject it

C)The local file can shadow the intended standard-library module

D)Python automatically renames the local file

Q24. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

```
print(fun())
```

A)10 only

B)Finally only

C)Finally followed by 10

D)10 followed by Finally

Q25. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)10

B)20

C)None

D)NameError

Q26. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():
```



```
nonlocal x
```

```
x = x + 5
```

```
return x
```

```
inner()
```

```
return x
```

```
print(outer())
```

A)5

B)10

C)15

D)UnboundLocalError

Q27. What is the output?

```
def fun(x):
```

```
    return x * 2
```

```
a = fun
```

```
print(a(5))
```

A)func(5)

B)5

C>Error

D)10

Q28. What happens when this code runs?

```
def first():
```

```
    second()
```

```
def second():
```

```
    third()
```

```
def third():
```

```
    raise RuntimeError("Failure")
```

```
try:
```

```
    first()
```

```
except RuntimeError:
```

```
    print("Handled")
```

A)RuntimeError cannot propagate through functions

B)TypeError

C)Handled

D)NameError

Q29. What is the output?

```
def outer():
```

```
    x = 10
```

```
def inner():
```

```
    return x + 5
```

```
return inner
```

```
f = outer()
```

```
print(f())
```

A)NameError

B)15

C)None

D)10

Q30. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
```

```
    lambda a, b: a * b,
```

```
    numbers
```

```
)
```

```
print(result)
```

A)10

B)9

C)0

D)24

Q31. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

A)A B

B)SyntaxError

C)A

D)B

Q32. What happens here?

```
def fun(a):
```

```
    return a
```

```
print(fun())
```

A)None

B)0



C)TypeError
D)NameError

Q33. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)B
B)A and B
C)A
D)Nothing

Q34. What will be printed?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

A)30
B)SyntaxError
C)TypeError
D)20

Q35. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)5
B)15
C)Error
D)10

Q36. What is the output?

```
def fun(x):  
    x += 10  
    return x
```

```
x = 5  
y = fun(x)
```

```
print(x, y)  
A)5 15
```

B)15 5
C)15 15
D)5 5

Q37. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

A)TypeError
B)25
C)15
D)SyntaxError

Q38. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)SyntaxError
B)30
C)20
D)TypeError

Q39. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)10
10
B)Module loaded
10
C)ImportError
D)Module loaded
Module loaded
10



Q40. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

A)10

Hello

B)x

display

C)Only Hello

D>Error

Q41. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

```
print(fun(3))
```

A)3

B)RecursionError

C)TypeError

D)6

Q42. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

A)ZeroDivisionError occurs before Exception

B)except blocks execute randomly

C)Exception is a superclass of ZeroDivisionError

D)ZeroDivisionError is not an exception

Q43. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)6

B)None

C)3

D)5

Q44. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

```
fun()
```

A)Nothing

B)B

C)A B

D)A

Q45. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A)NameError

B)20

C)30

D)10

Q46. What will be printed?

try:

```
    print(10 / 0)
```

except ZeroDivisionError:

```
    print("A")
```

except Exception:

```
    print("B")
```



A)Nothing

B)A

C)A B

D)B

Q47. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

A)Both import only sqrt

B)First provides the module namespace; second imports sqrt directly

C)First imports only sqrt

D)There is no difference

Q48. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A C

B)A

C)B

D)B C

Q49. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)25

B)sqrt(25)

C)5.0

D)5

Q50. What will be printed?

```
def make_counter():
```

```
    count = 0
```

```
    def counter():
```

```
        nonlocal count
```

```
        count += 1
```

return count

return counter

```
a = make_counter()
```

```
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)0

1

0

2

B)1

2

1

3

C)1

2

3

4

D)1

1

1

1

